

GitAlert 서비스 구조 문서

개요

GitHub(또는 Bitbucket)에서 Push 이벤트가 발생했을 때, **Windows 토스트 알림**으로 실시간 알려주는 프로그램입니다.

전체 서비스 흐름



핵심 구성 요소

1. GitHub Webhook

Webhook이란?

특정 이벤트가 발생했을 때 GitHub가 지정한 URL로 HTTP POST 요청을 자동으로 보내는 기능입니다. 우리가 주기적으로 API를 조회하는 게 아니라, GitHub가 먼저 알려주는 **Push 방식(이벤트 드리븐)** 입니다.

설정 위치

레포지토리 → Settings → Webhooks → Add webhook

설정 항목

항목	값	설명
Payload URL	https://xxx.ngrok-free.dev/webhook	이벤트를 받을 서버 주소
Content type	application/json	데이터 형식
Events	Just the push event	Push 이벤트만 수신

Webhook이 보내는 데이터 (Push 이벤트 JSON 예시)

```
{
  "pusher": { "name": "username" },
  "repository": { "name": "repo-name" },
  "ref": "refs/heads/main",
  "commits": [
    { "message": "커밋 메시지" }
  ]
}
```

2. ngrok (터널링)

왜 필요한가?

Webhook은 GitHub 서버가 내 PC로 HTTP 요청을 보내는 방식입니다. 그런데 일반 가정/사무실 PC는 **공인 IP가 없어서** 외부에서 직접 접근이 불가능합니다.

GitHub → 내 PC ← 직접 접근 불가 (공인 IP 없음)

ngrok은 중간에서 **공인 URL ↔ 로컬 서버** 를 연결해주는 터널 역할을 합니다.

GitHub → <https://xxx.ngrok-free.dev> (공인 URL) → localhost:5000 (내 PC)

실행 방법

```
ngrok http 5000
```

실행 시 출력

```
Forwarding https://xxx.ngrok-free.dev -> http://localhost:5000
```

⚠️ ngrok 무료 플랜은 실행할 때마다 URL이 바뀝니다. URL이 바뀌면 GitHub Webhook 주소도 다시 등록해야 합니다.

3. Flask 서버

Flask란?

Python의 경량 웹 프레임워크입니다. 여기서는 GitHub Webhook의 HTTP POST 요청을 받아서 처리하는 서버 역할을 합니다.

코드 구조 설명

```
from flask import Flask, request
from winotify import Notification

app = Flask(__name__) # Flask 앱 인스턴스 생성
```

`Flask(__name__)` 으로 앱을 생성합니다. `__name__`은 현재 파일명을 가리키며, Flask가 내부적으로 경로를 찾을 때 사용합니다.

```
@app.route('/webhook', methods=['POST'])
def webhook():
    ...
```

`@app.route`는 데코레이터입니다. `/webhook` 경로로 POST 요청이 오면 아래 `webhook()` 함수를 실행하라는 의미입니다.

```
data = request.json
```

GitHub가 보낸 JSON 데이터를 Python 딕셔너리로 파싱합니다.

```
if 'commits' not in data:
    return 'OK', 200
```

Push 이벤트가 아닌 경우(예: Webhook 등록 시 테스트 핑)는 그냥 200 OK만 반환하고 종료합니다.

```
pusher = data['pusher']['name'] # 푸시한 사람
repo = data['repository']['name'] # 레포 이름
branch = data['ref'].split('/')[ -1] # 브랜치 이름 (refs/heads/main → main)
commits = len(data['commits']) # 커밋 수
```

JSON에서 필요한 정보를 추출합니다. `data['ref']`는 `"refs/heads/main"` 형태라서 `.split('/')[ -1]`로 브랜치 이름만 꺼냅니다.

```
if __name__ == '__main__':
    app.run(port=5000)
```

`__name__ == '__main__'`은 이 파일을 직접 실행했을 때만 서버를 시작하라는 조건입니다. 다른 파일에서 import 할 때는 서버가 자동 실행되지 않습니다.

4. winotify (Windows 토스트 알림)

win10toast 대신 winotify를 쓰는 이유

win10toast는 내부적으로 pkg_resources의 구버전 API를 사용해서 Python 3.12+ 에서 호환 문제가 발생합니다. winotify는 이 문제가 없습니다.

```
toast = Notification(  
    app_id="GitAlert",           # 알림 출처 앱 이름  
    title="Push 알림",          # 알림 제목  
    msg="메시지 내용",          # 알림 본문  
    duration="short"            # 알림 표시 시간 (short / long)  
)  
toast.show()
```

실행 순서

1. 가상환경 활성화

```
.venv/Scripts/activate
```

2. Flask 서버 실행 (터미널 1)

```
python TestFlask.py
```

3. ngrok 실행 (터미널 2)

```
ngrok http 5000
```

4. GitHub Webhook 등록

ngrok이 출력한 URL을 GitHub Webhook Payload URL에 등록

5. Push 테스트

레포에 아무 파일이나 수정 후 Push → 토스트 알림 확인

사내 Bitbucket으로 전환 시

사내 Bitbucket은 보통 폐쇄망(인트라넷) 환경이라 ngrok을 통한 외부 터널링이 불가능합니다. 대신 같은 네트워크 안에서 로컬 IP로 직접 통신합니다.

변경 사항

```
# 기존
app.run(port=5000)

# 변경
app.run(host='0.0.0.0', port=5000) # 같은 네트워크에서 접근 허용
```

내 PC 로컬 IP 확인

```
ipconfig
# IPv4 주소: 192.168.x.x
```

Bitbucket Webhook 등록

Payload URL: `http://192.168.x.x:5000/webhook`

`host='0.0.0.0'`은 모든 네트워크 인터페이스에서 요청을 받겠다는 의미입니다. 이게 없으면 localhost에서만 접근 가능해서 Bitbucket 서버가 접근하지 못합니다.

패키지 목록

flask	웹 서버 프레임워크
winotify	Windows 토스트 알림

```
pip install flask winotify
```